1. Определение возраста покупателей

Перед нами стоит задача обучения нейросетевой модели для определения возраста покупателей по снимку их лица. У нас в распоряжении есть датасет с фотографиями людей, а также соответствующими им возрастами. За метрику мы берем MAE, пороговое значение - 8

1.1. Исследовательский анализ данных

Ввод [1]:

```
import pandas as pd
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
```

Ввод [2]:

```
labels = pd.read_csv('/datasets/faces/labels.csv')
datagen = ImageDataGenerator(rescale=1./255)
gen_flow = datagen.flow_from_dataframe(
        dataframe=labels,
        directory='/datasets/faces/final_files/',
        x_col='file_name',
        y_col='real_age',
        target_size=(224, 224),
        batch_size=32,
        class_mode='raw',
        seed=12345)
```

```
Found 7591 validated image filenames.
```

У нас имеется ~7600 изображений людей с разными возрастами. На каждого приходится в среднем по 76 фотографий, хотя распределение, конечно, иное. Исходя из того, что мы решаем задачу регрессии, это количество данных может быть недостаточным для корректного обучения. Тогда можно искусственно размножить изображения малочисленных возрастов: просто копировать их еще раз, растянуть, отразить и тд. Также, можно попробовать убрать фото людей в возрасте больше 85-90 лет, так как их очень мало, а их нахождение на кассе с каждым новым годом все более маловероятно

Ввод [3]:

```
labels.columns
```

Out[3]:

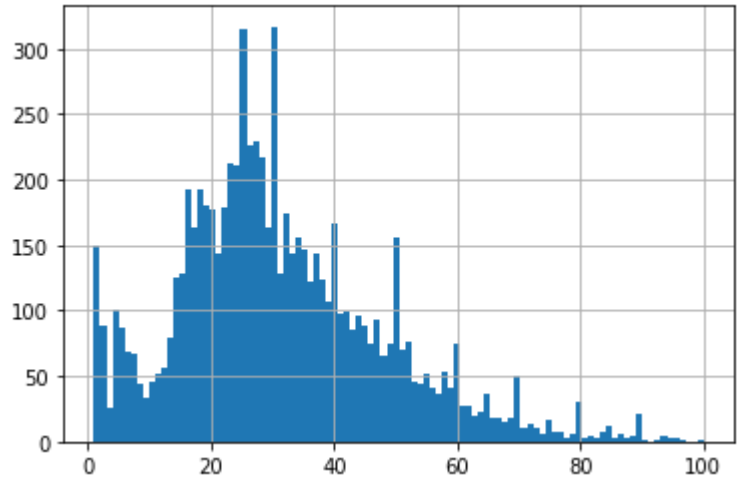
```
Index(['file_name', 'real_age'], dtype='object')
```

Ввод [17]:

```
labels.real_age.hist(bins=100)
```

Out[17]:

```
<AxesSubplot:>
```

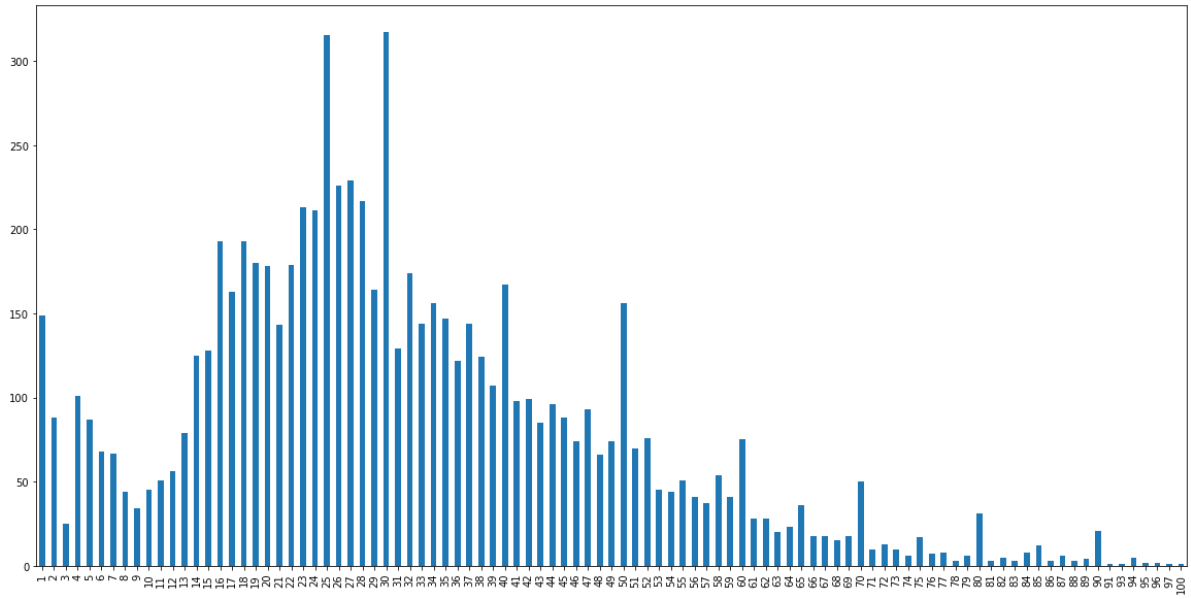


Ввод [16]:

```
labels.real_age.value_counts().sort_index().plot(kind='bar', figsize=(20, 10))
```

Out[16]:

```
<AxesSubplot:>
```



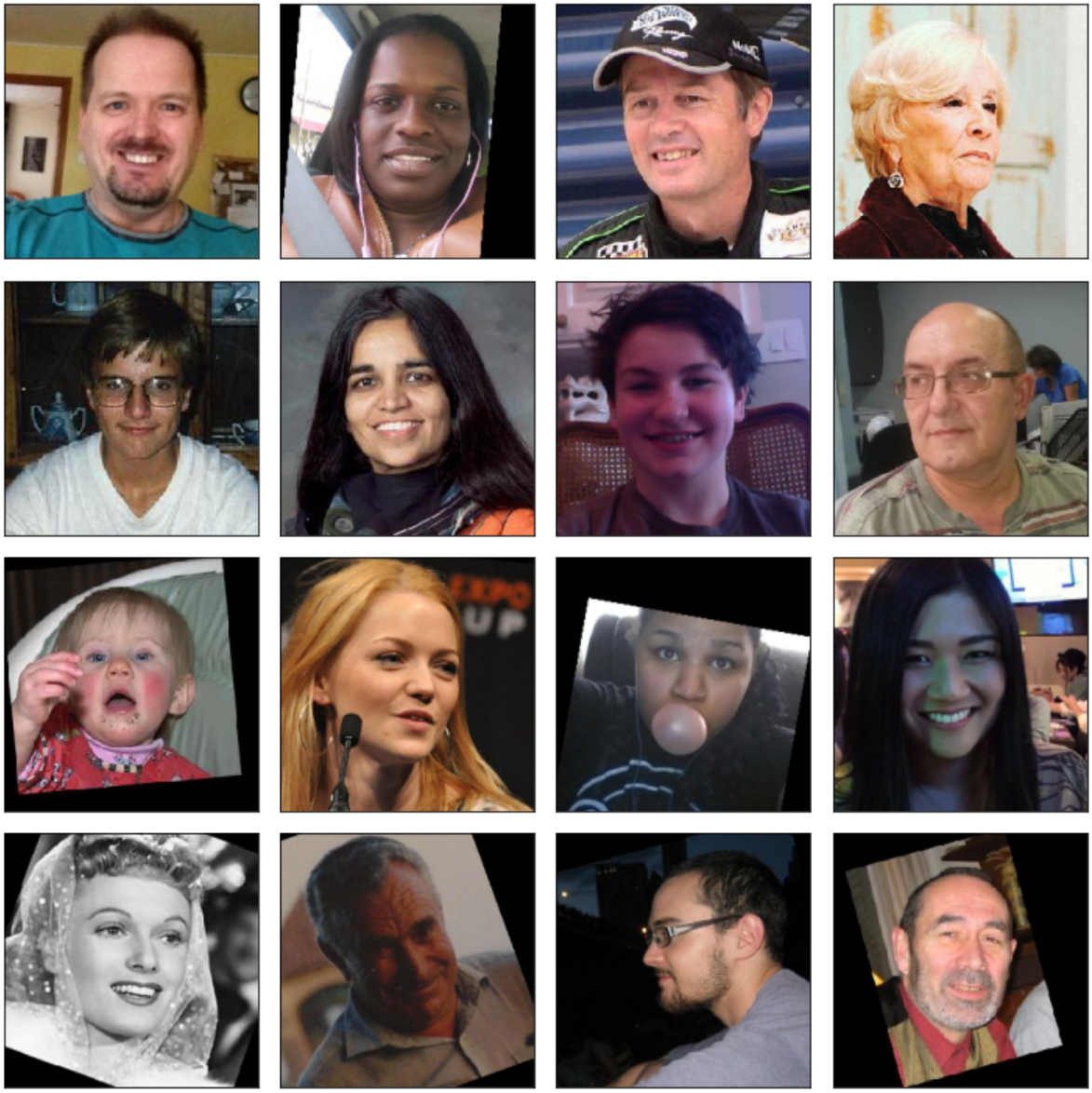
Распределение близко к нормальному, со смещением влево. Мы видим явное преобладание возраста 20-30 лет, с явными пиками на 25 и 30 годах. Также, есть просадка в районах 3 и 9 лет, а после 70 в целом фотографий очень мало, что затрудняет обучение

Ввод [11]:

```
features, target = next(gen_flow)
```

Ввод [12]:

```
fig = plt.figure(figsize=(10,10))
for i in range(16):
    fig.add_subplot(4, 4, i+1)
    plt.imshow(features[i])
    plt.xticks([])
    plt.yticks([])
    plt.tight_layout()
```



Фотографии разнообразные - здесь есть люди совершенно разных возрастов, а так же как семейные и личные фото, так и сцены из фильмов и публичные снимки. Условия съемки тоже различные, что добавляет баллов обученной модели (она сможет распознать людей в разных позах и при разном освещении)

1.2. Обучение модели

```python

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.datasets import fashion_mnist

from tensorflow.keras.applications.resnet import ResNet50

from tensorflow.keras.layers import Conv2D, Flatten, Dense, MaxPooling2D,
AvgPool2D, GlobalAveragePooling2D

from tensorflow.keras.models import Sequential

from tensorflow.keras.optimizers import Adam

import numpy as np

import pandas as pd




def load_test(path):



test_datagen = ImageDataGenerator(rescale=1./255,

validation_split=.25)



test_datagen_flow = test_datagen.flow_from_dataframe(dataframe =
pd.read_csv(path + 'labels.csv'),

directory= (path + '/final_files'),

x_col='file_name',

y_col='real_age',

target_size=(224, 224),

batch_size=32,

class_mode='raw',

subset='validation',
```

```
seed=12345)


return test_datagen_flow


def load_train(path):


train_datagen = ImageDataGenerator(rescale=1./255,

validation_split=.25,

horizontal_flip=True,

width_shift_range=0.1,

height_shift_range=0.1)


train_datagen_flow = train_datagen.flow_from_dataframe(dataframe =
pd.read_csv(path + 'labels.csv'),

directory= (path + '/final_files'),

x_col='file_name',

y_col='real_age',

target_size=(224, 224),

batch_size=32,

class_mode='raw',

subset='training',

seed=12345)


return train_datagen_flow
```

```
def create_model(input_shape):

model = Sequential()

backbone = ResNet50(input_shape=(224, 224, 3),
weights='/datasets/keras_models/resnet50_weights_tf_dim_ordering_tf_kernels_n
include_top=False)

model.add(backbone)

model.add(GlobalAveragePooling2D())

model.add(Dense(units=1, activation='relu'))

optimizer = Adam(lr=0.00001)

model.compile(optimizer=optimizer, loss='mean_squared_error', metrics=
['mean_absolute_error'])

return model


def train_model(model, train_data, test_data, batch_size=None, epochs=15,
steps_per_epoch=None, validation_steps=None):

model.fit(train_data,
validation_data=test_data,
```

```
epochs=epochs,

batch_size=batch_size,

steps_per_epoch=steps_per_epoch,

validation_steps=validation_steps,

verbose=2)


return model


```

```

Found 5694 validated image filenames.

Found 1897 validated image filenames.



Train for 178 steps, validate for 60 steps



Epoch 1/15


178/178 - 98s - loss: 737.6682 - mean_absolute_error: 22.5282 - val_loss:
1158.1391 - val_mean_absolute_error: 29.6154



Epoch 2/15
```

```
178/178 - 90s - loss: 199.3462 - mean_absolute_error: 10.6564 - val_loss:
1157.3562 - val_mean_absolute_error: 29.6039
```

```
Epoch 3/15
```

```
178/178 - 91s - loss: 112.3657 - mean_absolute_error: 7.9076 - val_loss:
948.9878 - val_mean_absolute_error: 26.1941
```

```
Epoch 4/15
```

```
178/178 - 90s - loss: 85.4450 - mean_absolute_error: 7.0121 - val_loss:
211.8060 - val_mean_absolute_error: 10.8238
```

```
Epoch 5/15
```

```
178/178 - 90s - loss: 72.2820 - mean_absolute_error: 6.4598 - val_loss:
111.8034 - val_mean_absolute_error: 8.1089
```

```
Epoch 6/15
```

```
178/178 - 93s - loss: 61.3580 - mean_absolute_error: 6.0614 - val_loss:
85.8058 - val_mean_absolute_error: 7.0548
```

```
Epoch 7/15

178/178 - 94s - loss: 53.8955 - mean_absolute_error: 5.6839 - val_loss:
87.9083 - val_mean_absolute_error: 6.9486

Epoch 8/15

178/178 - 95s - loss: 49.6815 - mean_absolute_error: 5.4535 - val_loss:
81.5800 - val_mean_absolute_error: 6.7494

Epoch 9/15

178/178 - 93s - loss: 45.6041 - mean_absolute_error: 5.2457 - val_loss:
77.8169 - val_mean_absolute_error: 6.6309

Epoch 10/15

178/178 - 90s - loss: 41.8541 - mean_absolute_error: 5.0028 - val_loss:
81.4241 - val_mean_absolute_error: 6.8345

Epoch 11/15

178/178 - 88s - loss: 37.4928 - mean_absolute_error: 4.7400 - val_loss:
76.7172 - val_mean_absolute_error: 6.6091
```

```
Epoch 12/15

178/178 - 88s - loss: 35.7163 - mean_absolute_error: 4.6597 - val_loss:
79.4912 - val_mean_absolute_error: 6.6761

Epoch 13/15

178/178 - 88s - loss: 33.9188 - mean_absolute_error: 4.4794 - val_loss:
79.7990 - val_mean_absolute_error: 6.6937

Epoch 14/15

178/178 - 88s - loss: 30.9207 - mean_absolute_error: 4.3134 - val_loss:
77.9624 - val_mean_absolute_error: 6.6400

Epoch 15/15

178/178 - 88s - loss: 29.6940 - mean_absolute_error: 4.2363 - val_loss:
76.5051 - val_mean_absolute_error: 6.5659

60/60 - 9s - loss: 76.5051 - mean_absolute_error: 6.5659
```

```
Test MAE: 6.5659
```

1.3. Анализ обученной модели

Модель обучилась на архитектуре ResNet50, в качестве алгоритма оптимизации был взят Adam с начальным шагом 0.00001. По эмпирическим данным, полученным в процессе подбора параметров модели, лучше всего она обучается именно на этой архитектуре, причем (видимо, из за качества изображений) шаг Adam'а равный 0.0001 не давал результатов на 3-5 эпохах: MAE не сдвигалась с места.

Также, сама модель ResNet50 была предобучена на подготовленных данных, мы же произвели ее адаптацию к задаче регрессии

Мы получили MAE, который составил 6.5659 на 15-й эпохе. Так что такую модель можно использовать